

SVG Icon Agent: A Collaborative LLM-Agent Pipeline for Editable SVG Icon Generation

Zhao Guanjie 23307130127

June 2026

Abstract

This report presents SVG Icon Agent, a lightweight multi-agent system that converts short English icon prompts into editable SVG icons. The project avoids training a raster image model and instead decomposes vector icon generation into goal management, prompt rewriting, planning, multi-candidate SVG generation, semantic critique, SVG-quality critique, consensus selection, optimization, validation, failure taxonomy, repair routing, refinement, memory curation, and artifact export. Local deterministic code is used only for prompt loading, memory retrieval, SVG syntax and safety checks, raster preview rendering, and reportable metrics. In the Web UI outputs used for this report, four icon-generation runs produced 12 candidate SVGs, 4 selected winners, 4 optimized baselines, and 4 final refined icons. The final refined validity rate was 100%, and the average validation score improved from 91.25 to 97.50 after repair. Code link: https://github.com/fwarinben/CG_PJ.

1 Introduction

Simple icons are a useful but demanding graphics target. They must be recognizable at small sizes, editable by designers, safe to embed in user interfaces, and compact enough to inspect. General text-to-image models can create attractive raster previews, but their outputs are difficult to edit as structured graphics. SVG, by contrast, stores geometry, color, and accessibility metadata as XML elements, making it a natural format for icon generation and downstream revision [World Wide Web Consortium, 2018].

SVG Icon Agent frames icon creation as an agentic graphics pipeline rather than a one-shot image generation problem. A user gives a short prompt, such as “a steaming cup of coffee suitable for an app icon.” The system retrieves similar local memories, creates explicit goals and acceptance criteria, rewrites the prompt, plans the icon, generates several SVG candidates, critiques them from semantic and SVG-quality perspectives, selects a winner, optimizes it, validates it, and repairs it when necessary. This design turns failure into an inspectable sequence of diagnoses and routed fixes instead of an opaque retry.

The project is positioned between vector graphics generation and language-agent workflows. It targets editable 256-by-256 SVG icons, but it uses LLM-backed agents to perform planning, critique, selection, and repair. The Web UI exposes this process through live workflow events, generated artifacts, validation logs, memory records, and an HTML gallery.

2 Related Work

Vector graphics generation. Prior work such as DeepSVG learns generative models over vector graphics sequences and hierarchical structure [Carlier et al., 2020]. These approaches are powerful when sufficient vector training data are available. SVG Icon Agent instead focuses on prompt-driven, LLM-backed SVG drafting and repair. It does not train a neural vector model; it constrains the model output to editable SVG primitives and uses deterministic checks to catch malformed or unsafe content.

Language agents and tool use. ReAct demonstrates that language models can combine reasoning traces with actions over tools and environments [Yao et al., 2023]. SVG Icon Agent follows a similar spirit by sepa-

rating model-backed reasoning stages from local tools. The SVG checker is not an agent; it is a deterministic tool that supplies evidence to SVG-quality, validation, and repair agents.

Self-refinement. Self-Refine and Reflexion show that language models can improve outputs by producing feedback, storing lessons, and iterating on failures [Madaan et al., 2023, Shinn et al., 2023]. SVG Icon Agent adapts this idea to vector graphics. The system validates generated SVG, classifies failures, routes repair strategies, and curates reusable memories from completed runs.

Engineering foundations. The implementation uses OpenRouter as the model API layer [OpenRouter, 2026], Pillow-related rendering utilities for lightweight image export [Pillow Contributors, 2026], and an ICLR-style LaTeX report format inspired by the public ICLR template repository [International Conference on Learning Representations, 2026]. The code is available in the project repository [SVG Icon Agent Project, 2026].

3 Method

3.1 System Overview

The pipeline is deliberately divided into model-backed agents and deterministic local tools. Every named agent calls the configured LLM provider. Local code provides the substrate: prompt parsing, memory retrieval, SVG checking, rendering, metrics, and gallery export. This split keeps the graphics logic inspectable while still allowing the LLM to handle semantic reasoning, design tradeoffs, and repair planning.

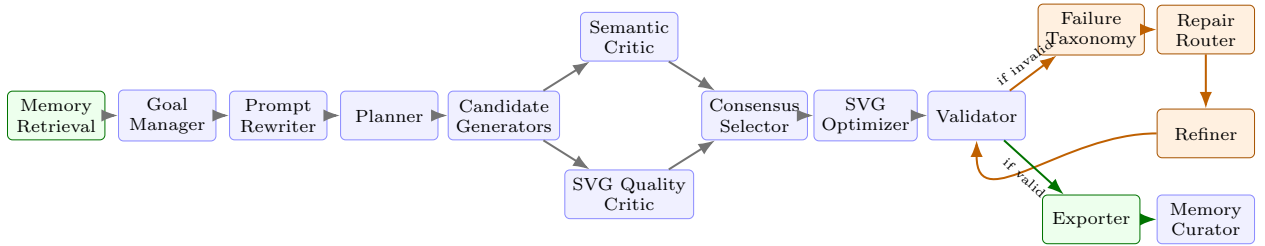


Figure 1: Directed Agent dependency graph matching the Web UI workflow display. Blue nodes are LLM-backed agents, green nodes are deterministic tools or exporters, and orange nodes are the conditional repair route. During Web runs, the same graph is colored by runtime status such as waiting, active, done, skipped, or error.

3.2 Agent Roles

Memory Retrieval Tool. Before generation, the system searches the local memory index for similar previous runs. Retrieved records contain summaries, success patterns, failure patterns, user feedback, scores, and tags. This stage is a tool rather than an LLM agent.

Goal Manager Agent. The goal manager converts the raw prompt, optional user goal, and retrieved memories into structured objectives, visual requirements, constraints, acceptance criteria, style preferences, and avoid patterns. For example, the coffee run produced constraints such as a 256-by-256 canvas, editable SVG format, no embedded raster images, and app-icon readability.

Prompt Rewriter Agent. The rewriter turns the short input into an SVG-icon-specific prompt. In the coffee run, the rewritten prompt requested a centered rounded cup body, a right-side handle, and three wavy steam lines while explicitly excluding saucers and beans.

Planner Agent. The planner outputs an icon plan containing category, style, palette, motifs, layout, and constraints. These plans make later validation more concrete. For the coffee icon, the plan specified a warm palette: #6F4E37, #D2B48C, and #F5F5DC.

Multi-Candidate Generator Agent. In collaborative mode, the generator creates three distinct SVG

candidates per prompt. This increases design diversity and gives the critics a real choice instead of forcing the pipeline to repair the first sample.

Semantic Critic Agent and SVG Quality Critic Agent. The semantic critic scores prompt alignment, recognizability, and small-icon readability. The SVG-quality critic scores editability, safety, viewBox correctness, primitive use, and rendering risk, using local SVG checker findings as evidence.

Consensus Selector Agent. The selector compares critic reports and chooses the candidate with the best balance of semantics and SVG quality. It also records risks and writes a repair brief for the optimizer. Across the four Web runs, selected winners were candidate 2 for both rocket prompts, candidate 3 for the cat prompt, and candidate 2 for the coffee prompt.

SVG Optimizer Agent. The optimizer rewrites the selected SVG using critic feedback, selector risks, local checker output, and optional manual feedback. The Web UI can also trigger a post-run optimizer pass without rerunning candidate generation.

Validator, Failure Taxonomy, Repair Router, and Refiner Agents. The validator evaluates the optimized baseline for semantic alignment, visual quality, editability, and rule compliance. If it fails, the failure taxonomy agent classifies the problem, the repair router chooses a route and ordered actions, and the refiner returns a complete revised SVG. This sequence makes repair explainable and auditable.

Memory Curator Agent. After completion, the curator summarizes successful strategies, failure patterns, user feedback, and scores into the local memory index. Later runs can retrieve these records as contextual guidance.

3.3 Deterministic Tools and Artifacts

The local SVG checker parses XML, enforces a 256-by-256 canvas and `viewBox`, rejects unsafe tags such as `script`, `foreignObject`, and `image`, flags external references, checks geometry attributes, and scores parseable SVGs based on errors and warnings. The exporter writes SVG files, PNG previews, validation reports, failure taxonomies, repair routes, LLM traces, raw response logs, metrics, and an HTML gallery.

This design keeps failures reproducible. A user can inspect validation reports, failure taxonomies, repair routes, raw LLM responses, metrics, and gallery files for every Web run.

4 Experiments

4.1 Data and Setup

The report uses the four completed Web UI runs under `outputs/web`. The prompts cover object-style icons: a minimal rocket launch icon, a minimal white rocket with blue flame, a cute gray smiling cat, and a steaming cup of coffee. Each run used collaborative mode with three candidates, memory retrieval enabled, prompt rewriting enabled, and OpenRouter-backed LLM calls. The model recorded in the Web output traces was `deepseek/deepseek-v4-flash`.

The evaluation is based on the JSON artifacts already produced by the system. Baseline refers to the optimized selected candidate before repair. Refined refers to the final SVG after zero or more repair rounds. The main metrics are validity, validation score, score delta, repair rounds, selected candidate, and repair route.

4.2 Quantitative Results

4.3 Qualitative Results

Figure 2 shows the final refined PNG previews exported from the SVG artifacts. The icons share the intended 256-by-256 format and use simple silhouettes appropriate for small UI usage.

Table 1: Aggregate results from the four Web UI runs in `outputs/web`.

Metric	Value
Number of prompts	4
Candidate SVGs generated	12
Selected winners	4
Baseline valid icons	3 / 4
Final refined valid icons	4 / 4
Baseline average score	91.25
Refined average score	97.50
Average score delta	+6.25
Maximum repair rounds used	1
Recorded LLM tokens	100,718
Recorded OpenRouter cost	\$0.0139

Table 2: Per-run validation and selection results.

Icon prompt	Winner	Baseline	Refined	Rounds	Main evidence
Minimal rocket launch icon	cand. 2	95, valid	95, valid	0	High semantic and SVG-quality scores; minor fin/window simplification risk.
Minimal white rocket launch	cand. 2	100, valid	100, valid	0	Correct white body, blue flame, centered readable composition.
Cute gray smiling cat	cand. 3	100, valid	100, valid	0	Strong cat identity, clear smile, safe primitive use.
Steaming coffee cup	cand. 2	70, invalid	95, valid	1	Semantic color mismatch repaired by <code>semantic_recompose</code> .

4.4 Repair Case Study: Coffee Icon

The coffee run demonstrates why an explicit repair architecture is useful. The selector chose candidate 2 because critics judged it balanced, editable, and visually clean. The post-run optimizer then applied manual feedback to make the cup blue. This improved user-directed color control but violated the planned coffee palette. The validator rejected the baseline with a score of 70 and reported invalid colors for the cup body, rim, and coffee surface, plus warnings about steam color and opacity.

The failure taxonomy agent classified the problem as `semantic_mismatch` and `constraint_violation`. The repair router selected the `semantic_recompose` route and produced ordered actions: replace blue fills with the planned brown/cream palette, change steam strokes to a palette color, remove opacity attributes, and update metadata. The refiner applied this brief, after which the validator accepted the refined SVG with a score of 95 and no issues. Figure 3 compares the invalid baseline and repaired output.

4.5 Discussion

The results are small but informative. Three of four optimized baselines were already valid, suggesting that the candidate-generation, critique, selection, and optimizer stages can produce high-quality SVGs without repair. The remaining case shows that the repair loop is not decorative: it diagnosed a real semantic constraint violation and restored validity in one round.

The memory subsystem also became active over time. The first rocket run had no retrieved records, while later runs retrieved one to three memories. These records sometimes came from different icon categories,

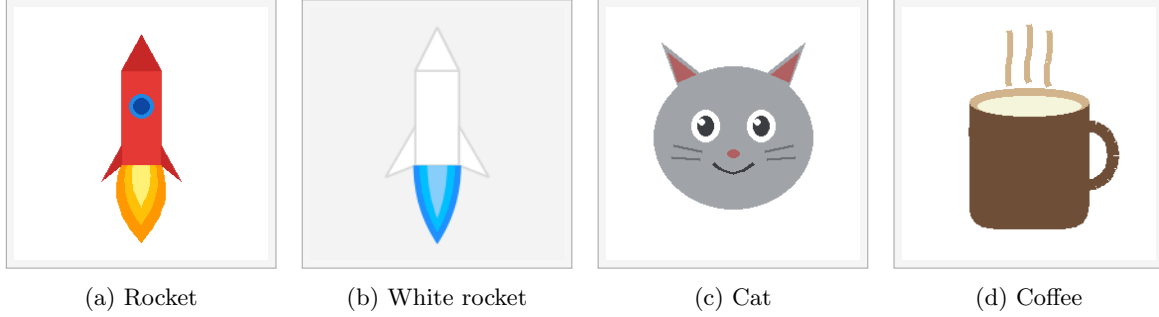


Figure 2: Final refined PNG previews generated from editable SVG outputs.

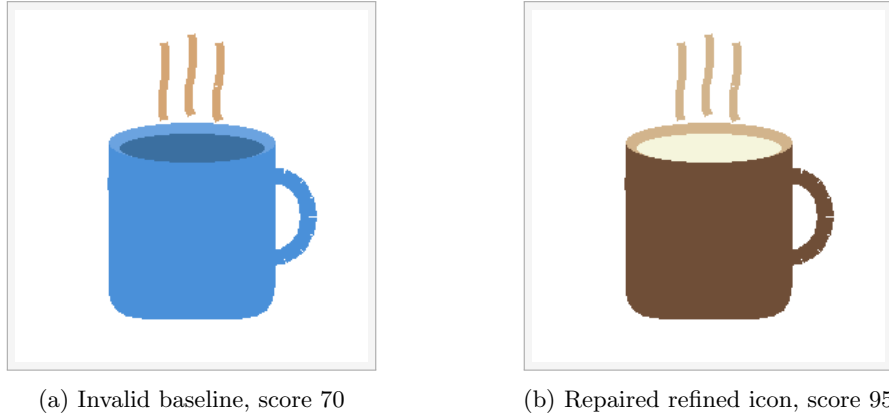


Figure 3: Coffee repair case. The agent loop converts a validation failure into a taxonomy, a route, ordered actions, and a valid revised SVG.

which is useful for generic layout lessons but can also introduce weakly related context. Future work should improve retrieval filtering so object-specific memories are preferred when semantic overlap is high.

The Web UI is important for evaluation because it exposes both artifacts and reasoning traces. A user can see the prompt rewrite, goal, plan, candidate SVGs, selected winner, optimizer feedback sources, validation reports, repair route, raw LLM responses, and final gallery. This makes the agent architecture easier to debug than a single hidden model call.

5 Conclusion

SVG Icon Agent demonstrates that editable icon generation can be organized as a collaborative LLM-agent pipeline rather than a monolithic raster generation task. The system separates semantic design work from deterministic SVG checking, uses multiple critics and a selector for explainable candidate choice, and repairs invalid outputs through a taxonomy-router-refiner loop. On four Web UI runs from `outputs/web`, it generated valid final SVG icons for every prompt and improved the average validation score from 91.25 to 97.50.

The main limitations are the small evaluation set, dependence on external LLM API reliability, and the current SVG checker’s practical but incomplete model of visual quality. Future work should add larger prompt suites, human preference evaluation, stronger memory retrieval, and ablations comparing collaborative mode against the single-shot workflow.

References

- Alexandre Carrier, Martin Danelljan, Alexandre Alahi, and Radu Timofte. DeepSVG: A Hierarchical Generative Network for Vector Graphics Animation. In *Advances in Neural Information Processing Systems*, 2020.
- International Conference on Learning Representations. ICLR Master Template. <https://github.com/ICLR/Master-Template>, 2026. LaTeX template repository.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, 2023.
- OpenRouter. OpenRouter API Documentation. <https://openrouter.ai/docs>, 2026. API documentation.
- Pillow Contributors. Pillow Documentation. <https://pillow.readthedocs.io/>, 2026. Python Imaging Library documentation.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, 2023.
- SVG Icon Agent Project. CG_PJ: SVG Icon Agent. https://github.com/fwarinben/CG_PJ, 2026. Project code repository.
- World Wide Web Consortium. Scalable Vector Graphics (SVG) 2. <https://www.w3.org/TR/SVG2/>, 2018. W3C Candidate Recommendation.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023.